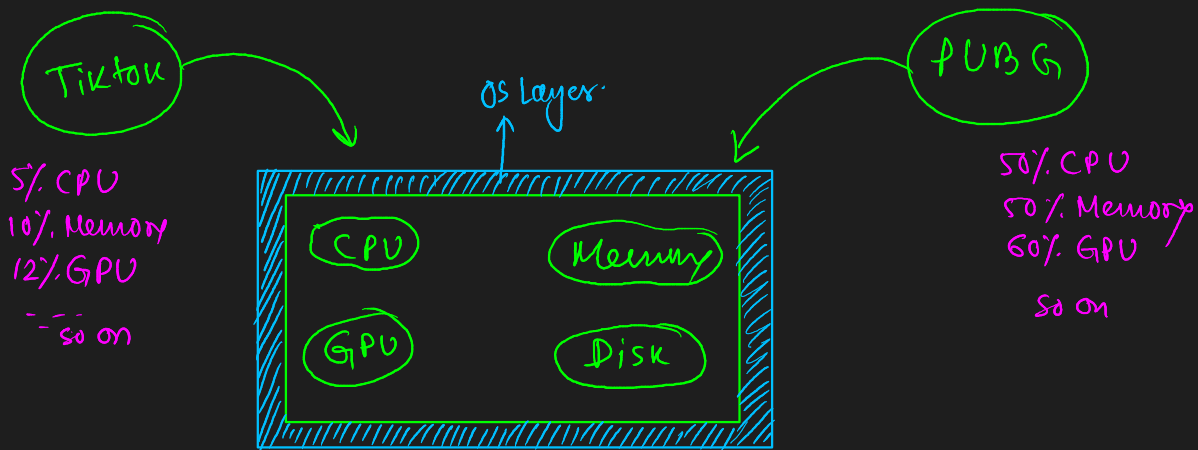
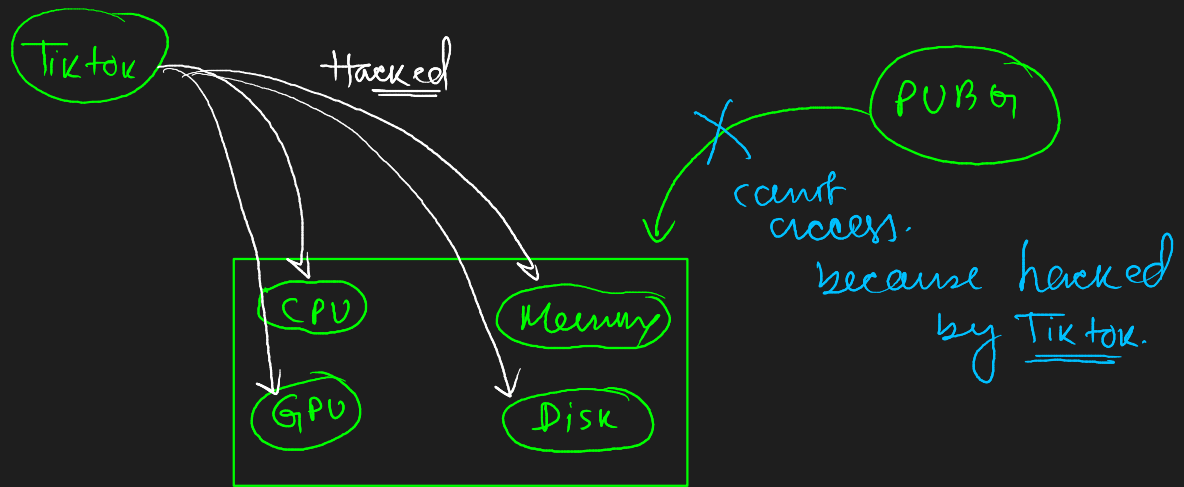


23/06/26

OS

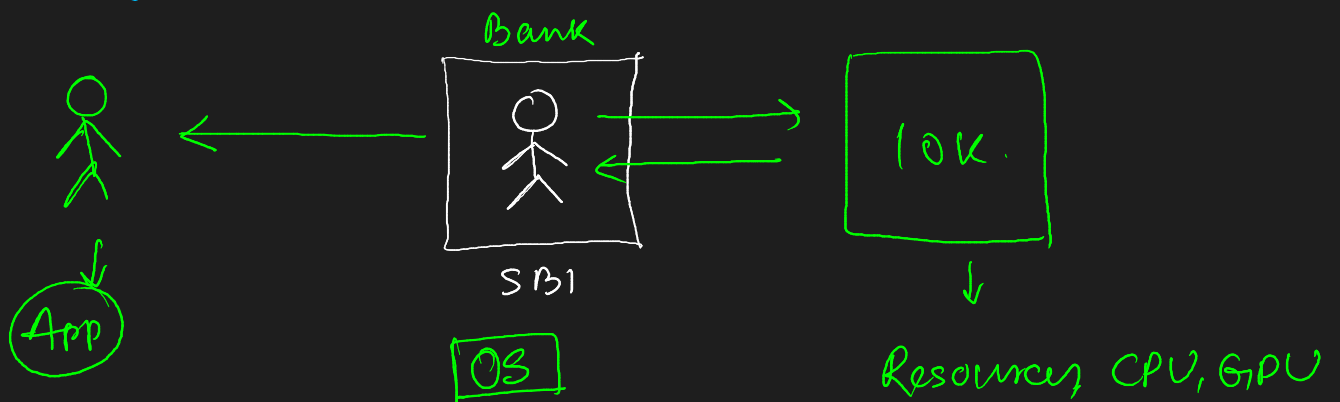
Lec-1

What happen if there is no OS?



OS: Resource Management

OS: Interface.



C++ → Malloc, Java → new

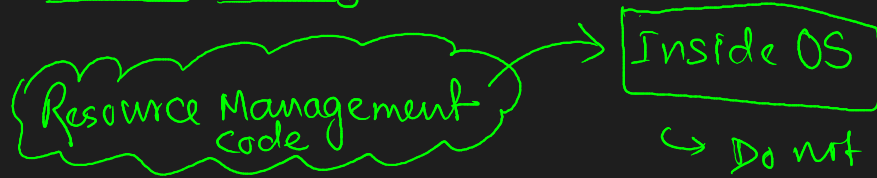
TikTok → Memory Management code (700mb)
→ App Logic code (50mb)
→ Bulky

PUBG → Game Logic code
→ Memory Management
→ Repeating code.

extra:

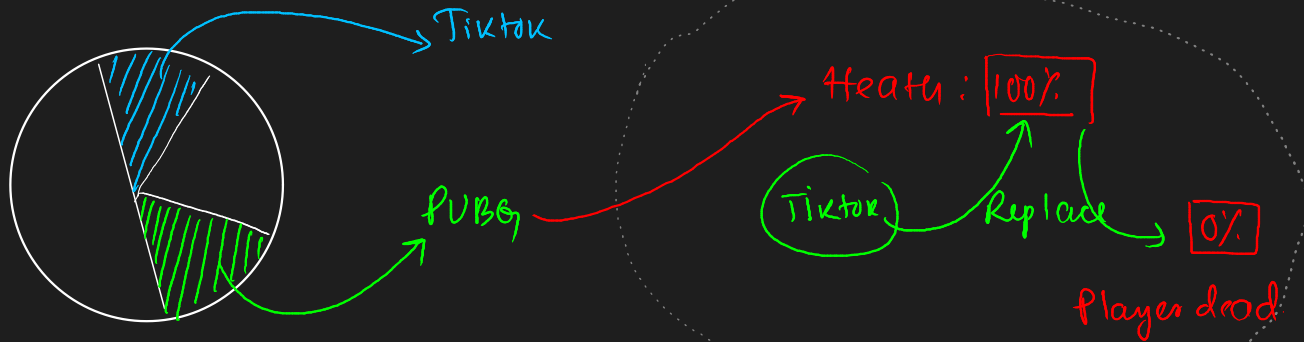
DRY → Do not Repeat Yourself

Prime fn

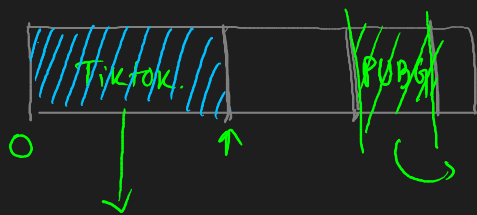


→ Do not Repeat in every application.

* Isolation And Protection



If OS exist:



Tiktok do not know there is running PUBG.

If NO OS

BIG PROBLEM !!

→ PUBG do not know that Tiktok is running.

- The 2 applications have their separate existence.
- Provides isolation.

■ If there is no OS?

- Bulky and complex app.
- Resource exploitation by 1 app.
- No memory protection.

■ OS Function —

- ① Resource Management (Aka, Arbitration)
- ② Interface b/w user & comp. hardware.
- ③ Abstraction - hides complexity
- ④ Isolation & protection.
- ⑤ Access to the computer hardware.

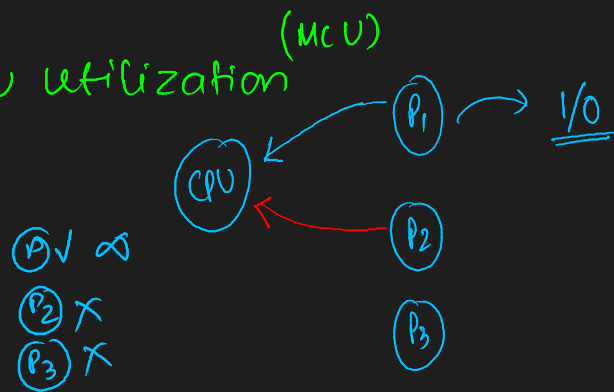
■ **[OS]** → An OS is a piece of software that manages all the resources of a computer system, (hardware + software) and provides an environment in which the user can execute his/her programs in a convenient and efficient manner by hiding underlying complexity of the hardware and acting as a resource manager.

Lee-2

Goals of OS → ① Maximum CPU utilization (MCU)

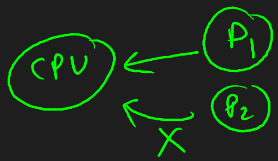
② Process Starvation (PS) X

③ High Priority Execution



Types of OS

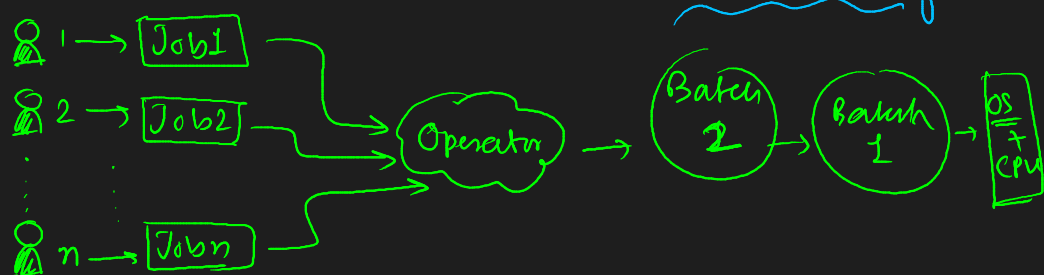
① Single Process OS



MCU X
PS ✓
High Priority X

Eg: MS DOS.

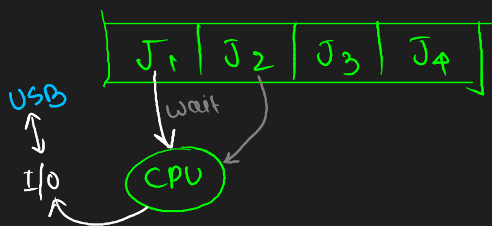
② Batch Processing OS



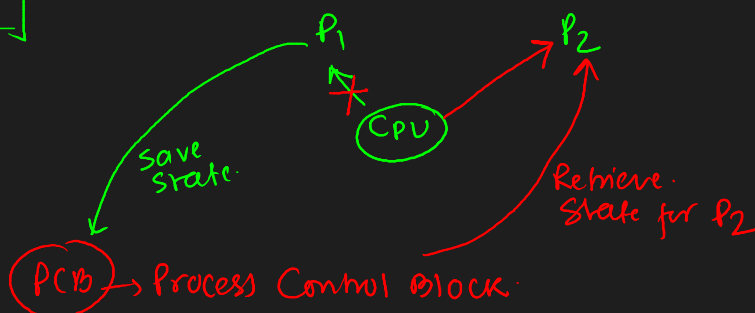
MCU X
PS ✓
High Priority X

③ Multi Programming OS

→ single CPU.



Context Switching



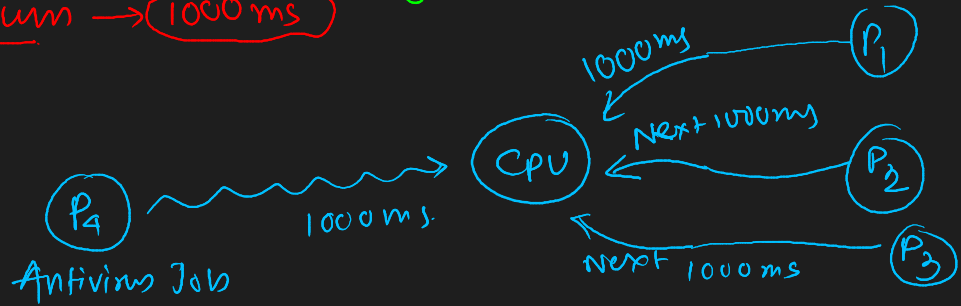
PCB → Process Control Block.

④ Multi Tasking OS

- Single CPU
- Context Switching
- Time Sharing $\Rightarrow *$

MCU $\checkmark$
PS $\times$
HighPrio $\checkmark$

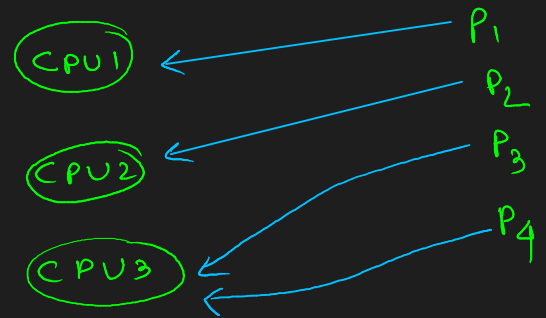
Time Quantum $\rightarrow$ 1000ms



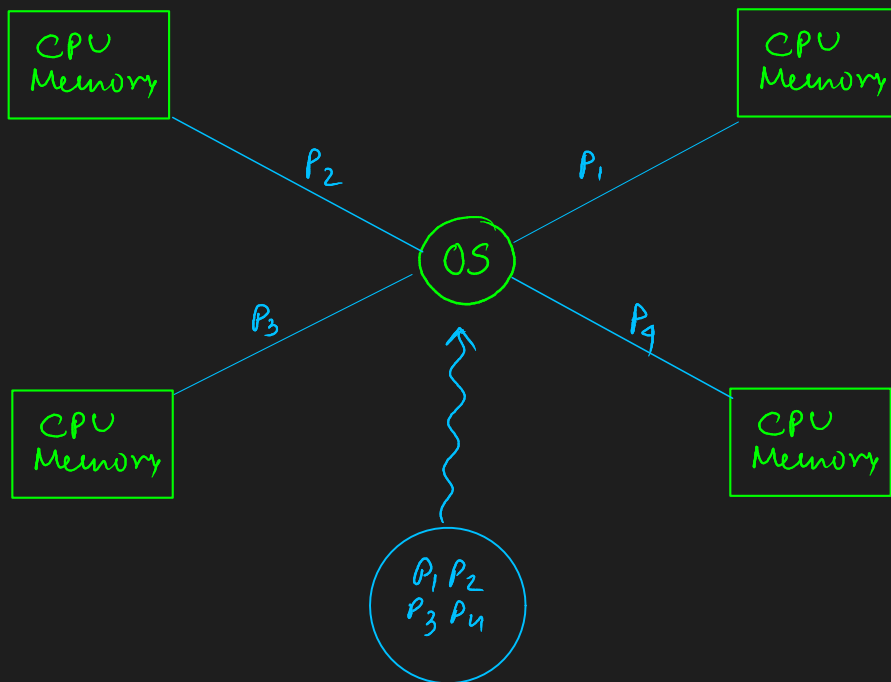
⑤ Multi processing OS

- Context Switching $\checkmark$
- Time Sharing $\checkmark$
- CPU ≥ 1

[increased Reliability]



⑥ Distributed OS - loosely coupled, autonomous, interconnected



⑦ RTOS - Real Time OS → Error Chances ↓↓
 - Realtime.

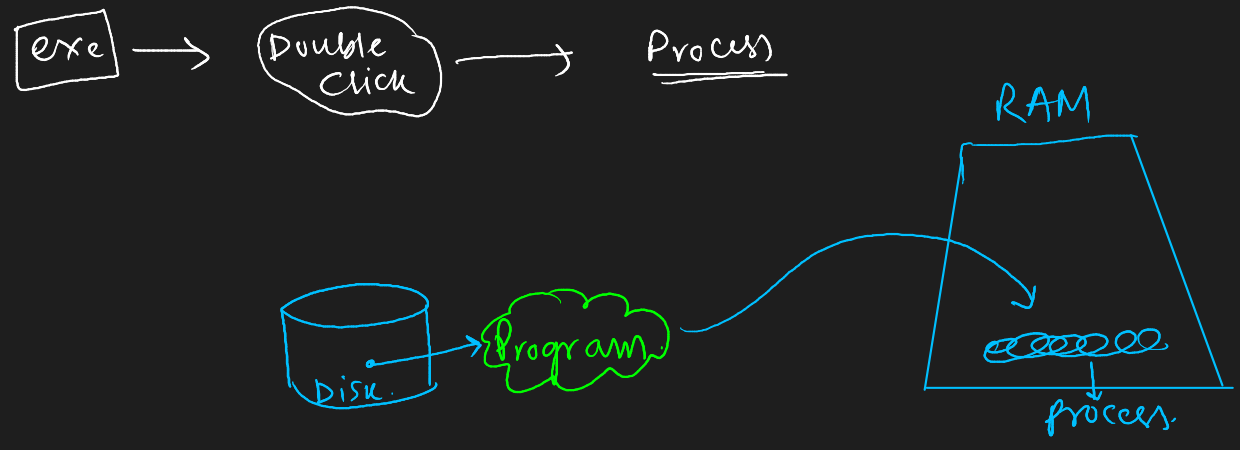
Usage
 → ATC (Air Traffic Control)
 → Industrial Apps

Ex. Fig.

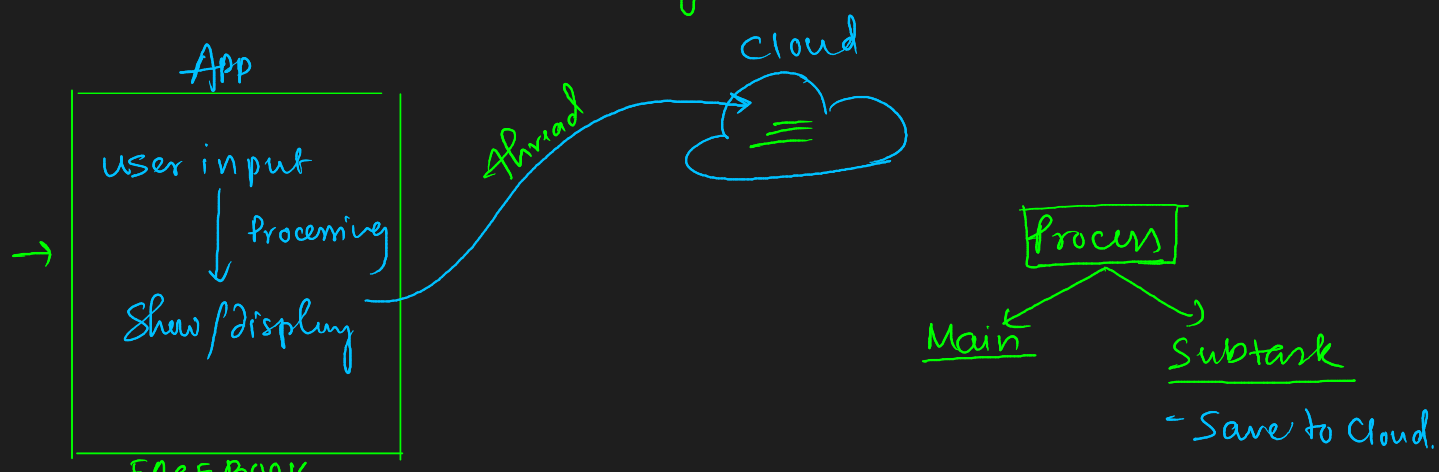
- ① Single Process → Ms DOS
- ② Batch Process → ATLAS
- ③ Multi programming → THE
- ④ Multi Tasking → CTSS
- ⑤ Multi processing → 
- ⑥ Distributed → Amoeba, Plan 9, LOCUS
- ⑦ RTOS → FreeRTOS, VxWorks, RT-Linux.

lec-3

Process → Program under Execution

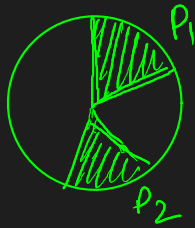


Thread → Lightweight process
 → Independently Executes.



Multi Tasking

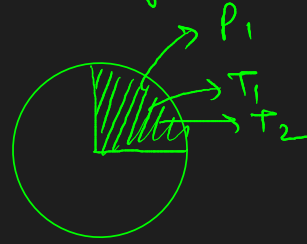
- ① ≥ 1 processes concept.
- ② Isolation & Memory Protection Concept.



- ③ Process Scheduling

Multi Threading

- ① ≥ 1 threads.
- ② No concept.



- ③ Thread Scheduling

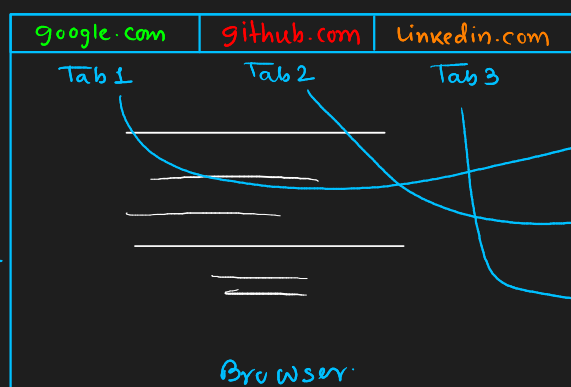
Program: A program is an executable file which contains a certain set of instructions written to complete the specific job or operation on your computer.

- Simplified code, ready to be executed.
- Stored in Disk.

Process: Program under execution. Resides in computer's primary memory (RAM).

Thread:

- Single Sequence Stream within a process.
- An independent path of Execution in a process.
- Light-weight process.
- Used to achieve parallelism by dividing a process's tasks which are independent path of execution.
- Eg: Multiple Tabs in a browser, text editor.



3 different
Threads
Executing simultaneously

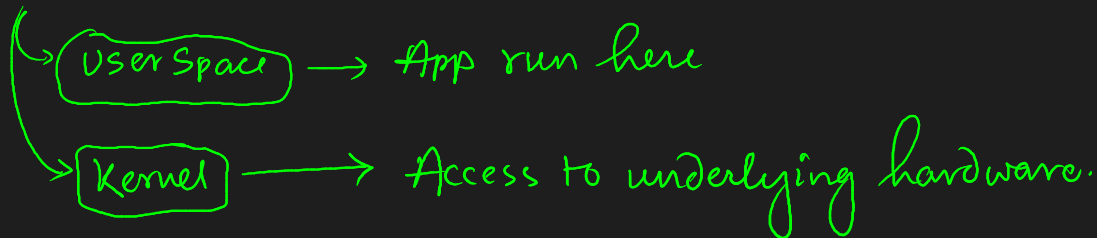
A single
Process

Thread Scheduling: Threads are scheduled for execution based on their priority. Even though threads are executing within the runtime, all threads are assigned processor time sliced by the OS.

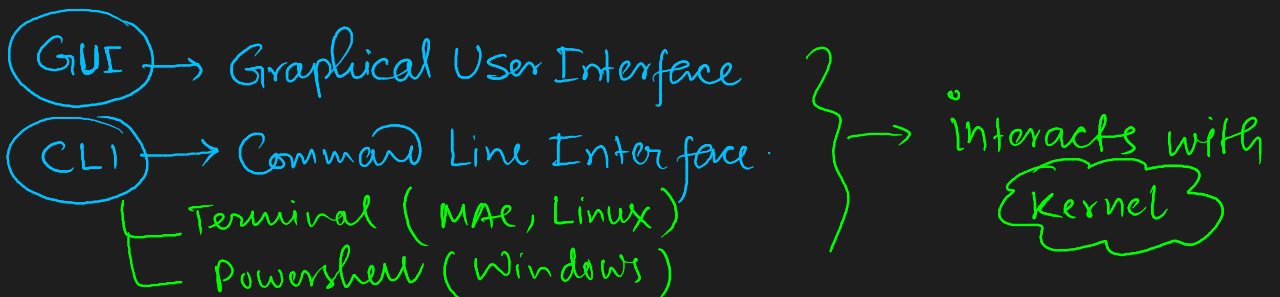
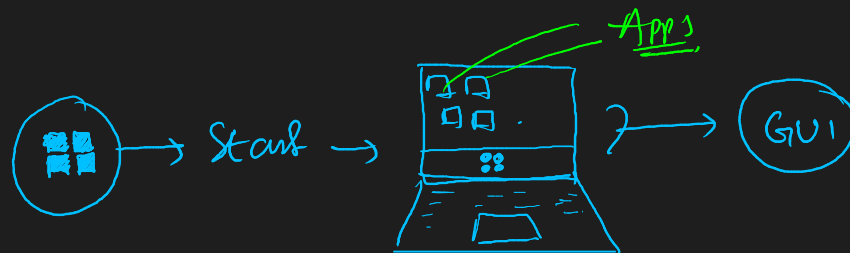
Thread Context Switch (TCS)	Process Context Switch (PCS)
TCS occurs when the CPU saves the current state of the thread and switches to another thread of the same process.	PCS occurs when the operating system's scheduler saves the current state of the running Program (including the state of PCB) and switches to another program.
TCS helps the CPU to handle multiple threads simultaneously.	PCS involves loading of the states of the new program for its execution.
TCS does not involve switching of memory address spaces. All the memory addresses that the processor accounts remain saved.	PCS involves switching of memory address spaces. All the memory addresses that the processor accounts get flushed.
Processor's cache and Translational Lookaside Buffer preserves their state.	Processor's cache and TLB gets flushed.
Though TCS involves switching of registers and stack pointers, it does not afford the cost of changing the address space. Hence it is more efficient.	PCS involves the heavy cost of changing the address space. Hence it is less efficient.
TCS is a bit faster and cheaper.	PCS is relatively slower and costlier.

lec-4

Components of OS

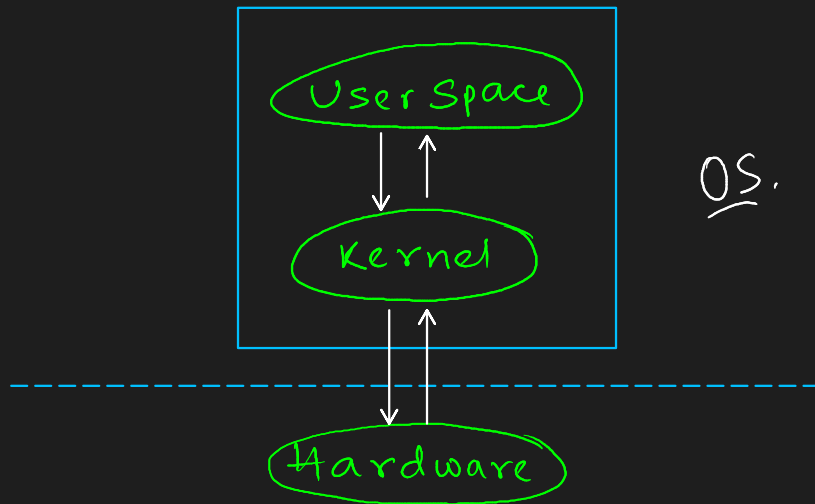


- ① **User Space** → No hardware access
 → convenient environment user apps.
- GUI**

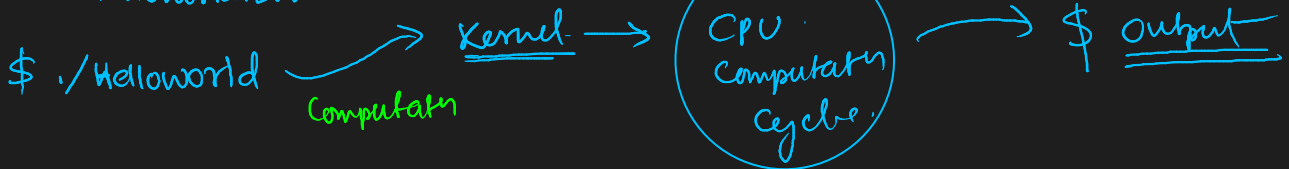


② Kernel → ♥ of OS.

→ Interacts with Hardware.



eg. `helloworld.sh`



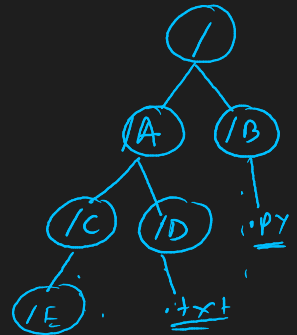
Kernel Function.

① Process management — Process creation
— termination
— Schedule Process & Threads.
— Process Synchronize
— Process Communication.

② Memory Management — Allocate/deallocate
— Free Space Management
—

③ File Management — create/delete
— directory management.

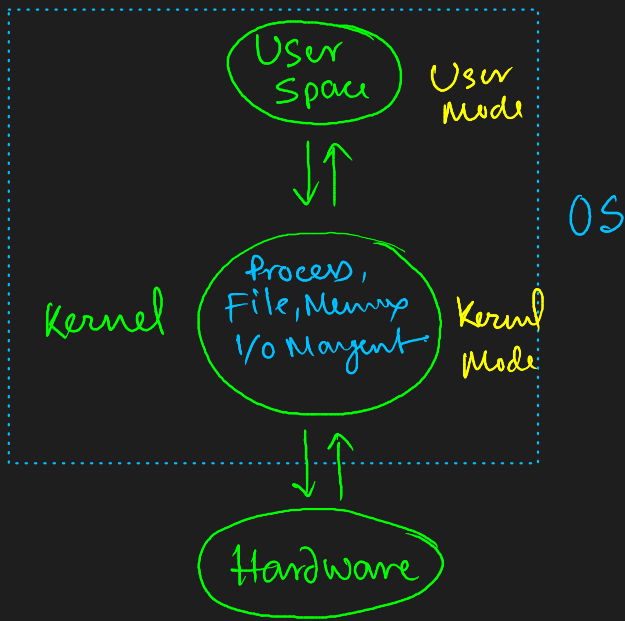
④ I/O Management — Management & controlling of all I/O devices.



4.1 Spooling
4.2 Buffering
4.3 Caching

Types of Kernel

① Monolithic Kernel



Adv.

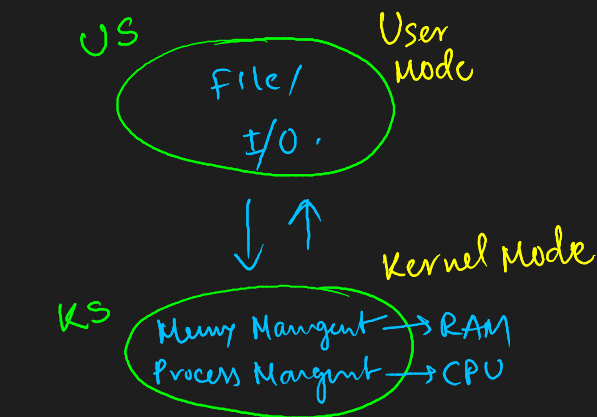
- ① Fast communication b/w different components of kernel.

DisAdv

- ① Bulky kernel.
- ② Less reliable.

Eg. Linux, Ms, DOS.

② Micro Kernel



Adv.

- ① Less Bulky.
- ② More Reliable.
- ③ More Stable.

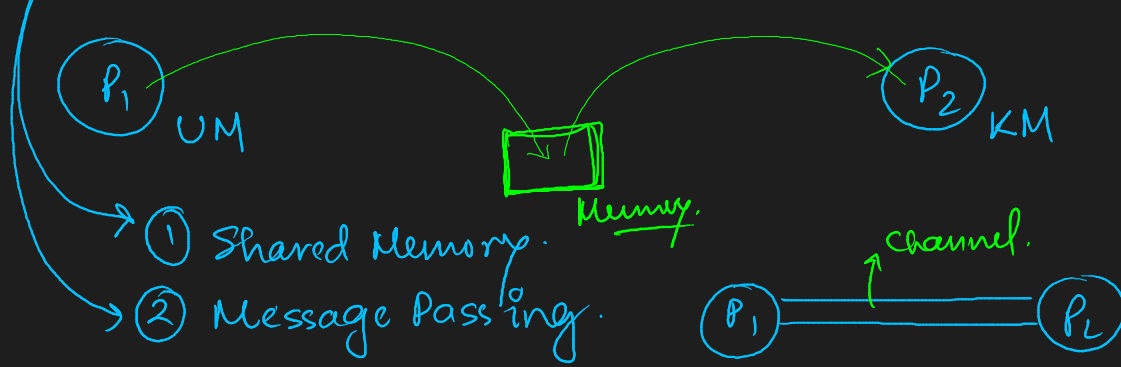
DisAdv

- ① Performance ↓↓
- ② Overhead ↑ UM/KS

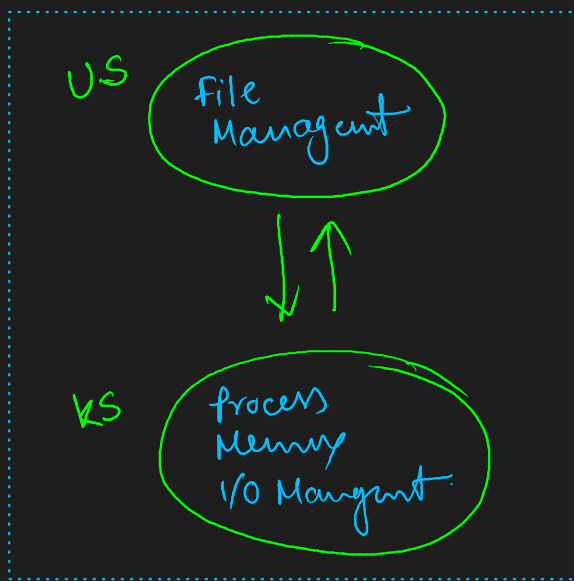
Eg. L4-linux, Symbian OS.

Q. UM & KM - how they communicate?

IPC → InterProcess Communication.



③ Hybrid Kernel - combined Approach.



Eg. MacOS,
Windows NT.
(7+)

④ Nano kernel } Others
⑤ Exo kernel.

24/06/26
lec-5

System calls → [App interact with Kernel]

SCI → System Call Interface

AIM → need folder creation - movies

✓ GUI → easy -

New Folder

Button.

✓ CLI → cmd →

mkdir

US

mkdir

KS

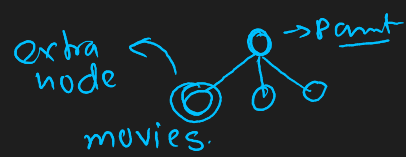
SCI

Hardware

Implementation

run

mainly implemented in C



system calls → C ✓

① process → HelloWorld.exe → Run

(→ System call execution.

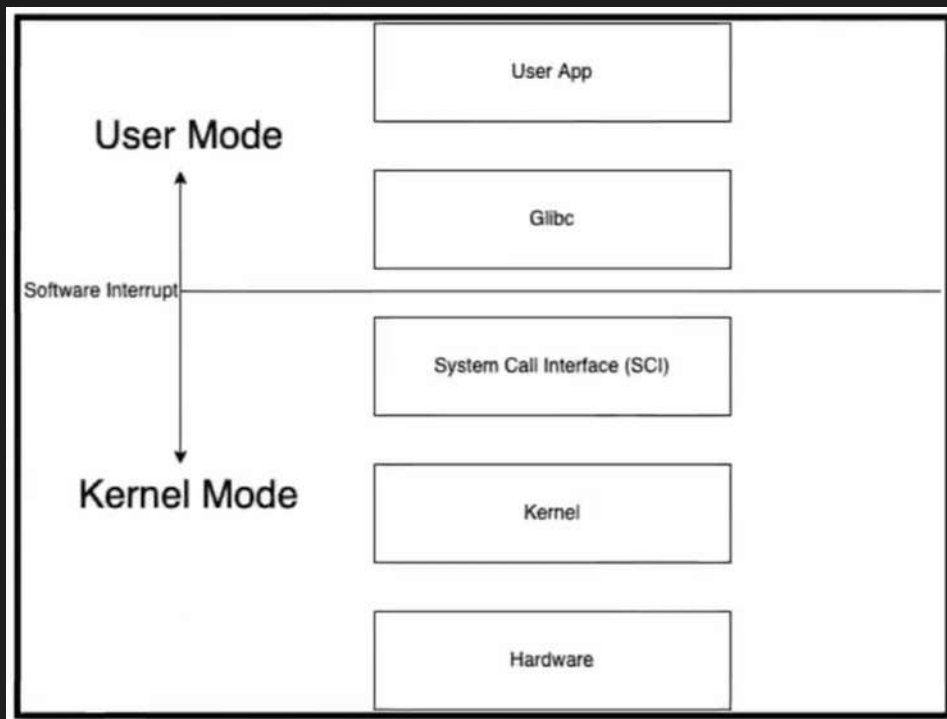
(→ KM switch. → Process Management
→ create process.

→ KM → UM switch.

A **system call** is a mechanism using which a user program can request a service from the kernel for which it does not have the permission to perform.
User programs typically do not have permission to perform operations like accessing I/O devices and communicating other programs.

UM ↔ KM switch

→ using software interrupt.



System calls are the only way through which a process can go into Kernel Mode from User Mode.

KM → UM

25/06/26

Types of System Calls

① Process Control

- end, abort, allocate memory
- load, execute, wait, signal etc.

② File Management

- create, delete, open
close file

③ Device Management

- request device, release device.
- read, write, reposition.
- get/set attributes.

④ Information Maintenance

→ get time/date
set time/date

- get process file or device attributes.

⑤ Communication Management:

- create, delete communication management
- send, receive message
- transfer status info.

System calls in Windows & Unix.

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Practical Example

▲touch hello.sh

```
open()  
write()  
close()
```

```
hello.sh
```

```
8 #!/bin/bash
7 while [ true ]
6 do
5     echo "hello world"
4     sleep 1
3 done
2
```

Execute Script

```
▲ bash hello.sh
```

```
hello world
hello world
hello world
hello world
hello world
hello world
hello world
```

```
fork()  
execve()  
open()  
read()  
close()
```

cheek p.id.

```

PS -a
PID TTY          TIME CMD
24195 ttys000      0:00.02 /usr/bin/login -flp anikchand /bin/zsh -fc exec -a -zsh /bin/zsh
24196 ttys000      0:02.55 -zsh
24243 ttys000      0:00.00 -zsh
24244 ttys000      0:00.18 -zsh
35859 ttys000      0:00.04 bash hello.sh
35934 ttys000      0:00.01 sleep 1
13802 ttys001      0:43.23 /bin/zsh
13855 ttys001      0:00.00 /bin/zsh

```

getpid() → 35859
read()
write()

getpid() → 35859
read()
write()



```
kill(  
[SIGKILL]  
exit()
```

```

# ps -al

```

UID	PID	PPID	F	CPU	PR	NI	SZ	RSS	WCHAN	S	ADDR	TTY	TIME	CMD
0	24195	24194	4066	0	31	0	435308896	4624	-	Ss	0	ttys000	0:00.02	/usr/bin/login -flp antikhand /bin/
501	24196	24195	4066	0	31	0	435313744	2112	-	Ss	0	ttys000	0:02.55	-zsh
501	24242	1	6	0	31	0	435306384	288	-	Ss	0	ttys000	0:00.00	-zsh
501	24244	1	6	0	31	0	435306368	1008	-	Ss	0	ttys000	0:00.18	-zsh
501	35859	24196	4006	0	31	0	435300928	6352	-	S+	0	ttys000	0:00.05	bash hello.sh
501	35945	35859	4006	0	31	0	435299488	1120	-	S+	0	ttys000	0:00.01	sleep 1
501	13802	13801	4006	0	31	0	435311440	3760	-	Ss+	0	ttys001	0:43.23	/bin/zsh

pid of the process
hello.sh.

pid of the
parent process (zsh)

[illegible]

lec-6 What happens when open computer? (Boot process)

5 steps

① Power ON.

(ON)

→ Power supply
→ Motherboard
→ Harddisk or other devices.

② CPU Loads BIOS or UEFI

BIOS = Basic Input Output System

UEFI = Unified Extensible Firmware Interface.

BIOS

Non Volatile Chip (ROM)

1 CPU Initialize

2 go to a chip BIOS

③ BIOS/UEFI Run tests & init hardware.

1 Loads some settings from a memory area.

backed by CMOS Battery

2 Program loads with settings.

POST → Power On Self Test.

④ BIOS/UEFI hands off to Boot device

Boot devices →

Disk (HDD/SSD)
CD
USB device

Boot loader → Program Execute on actual OS

EFI → Partition in disk
↓
Bootloader.

⑤ Bootloader Loads the Full OS

program → Actual OS initialize.



→ bootmgr.exe



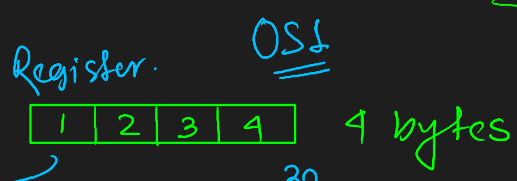
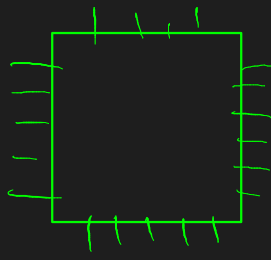
→ boot.efi

LINUX

→ GRUB



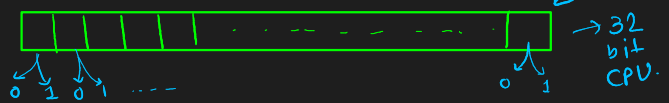
32 bit vs. 64 bit OS.



00000000

32 bit CPU

1 byte = 8 bits



2^{32} → unique values can locate } 32 bit CPU.

4GB RAM

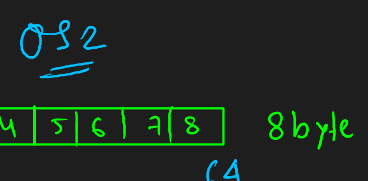
1 address = 1 byte

∴ can store 2^{32} unique addresses.

∴ 2^{32} byte = 4GB

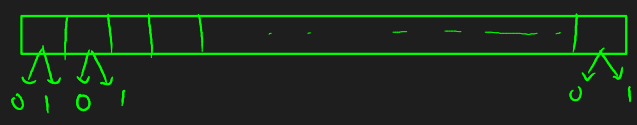


Address memory



8 bit.

64 → 8x8 bit CPU

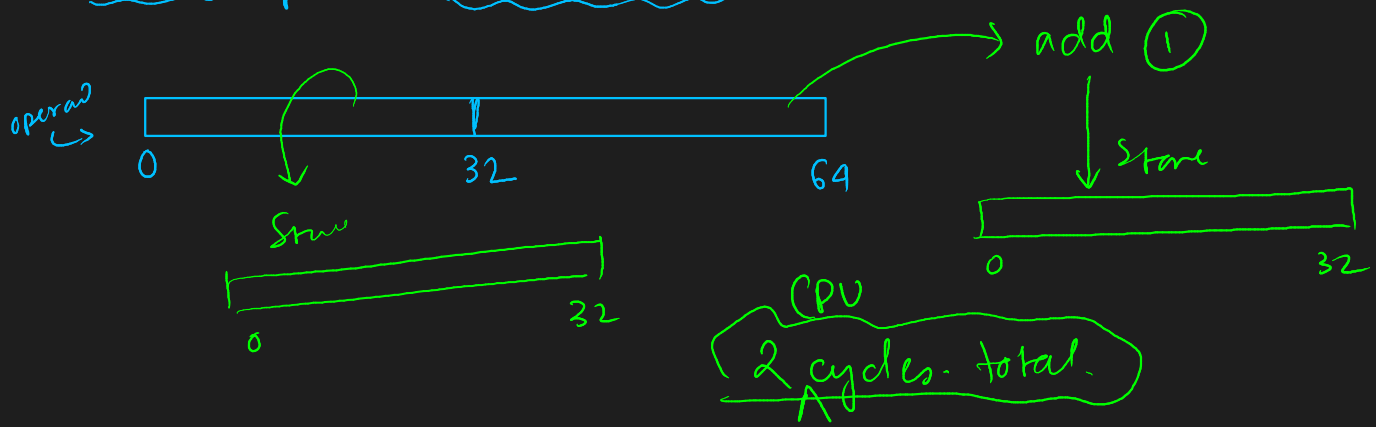


64 bit CPU

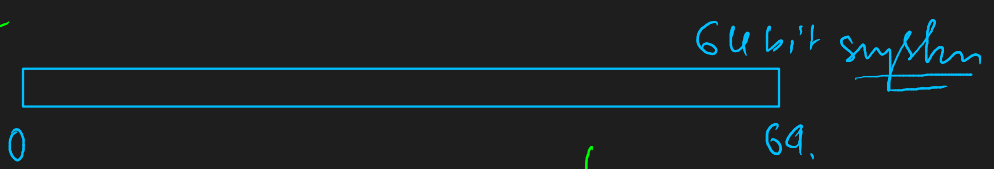
2^{64} byte = 1.845×10^{10} GB

RAM

32 bit system 64 bit number add.



but

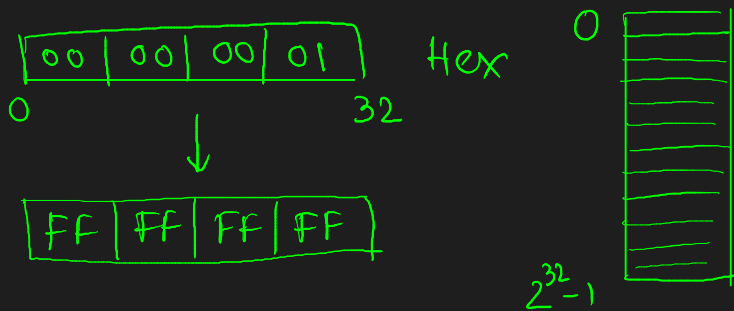


Single CPU cycle to operate 64 bit number

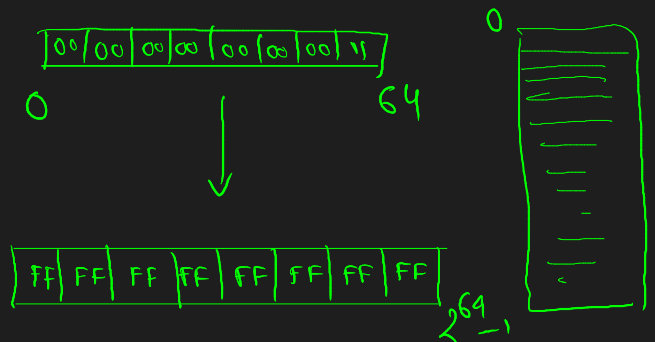
Advantages: →

① Addressable spaces.

32 bit CPU → 2^{32} diff. locations.



64 bit CPU → 2^{64} diff.



② Resource usage

③ Performance ↑ in 64 bit

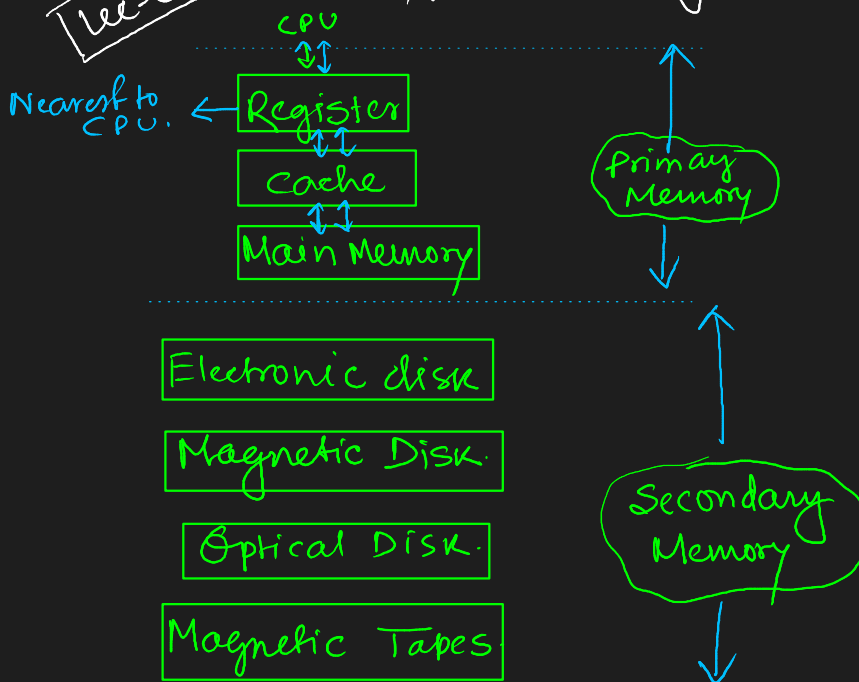
④ Compatibility → 64 bit supports → 64 & 32 bit both
→ 32 bit supports → only 32 bit

⑤ Better graphics performance.

28/06/26

Lee-8

Types of Storage.



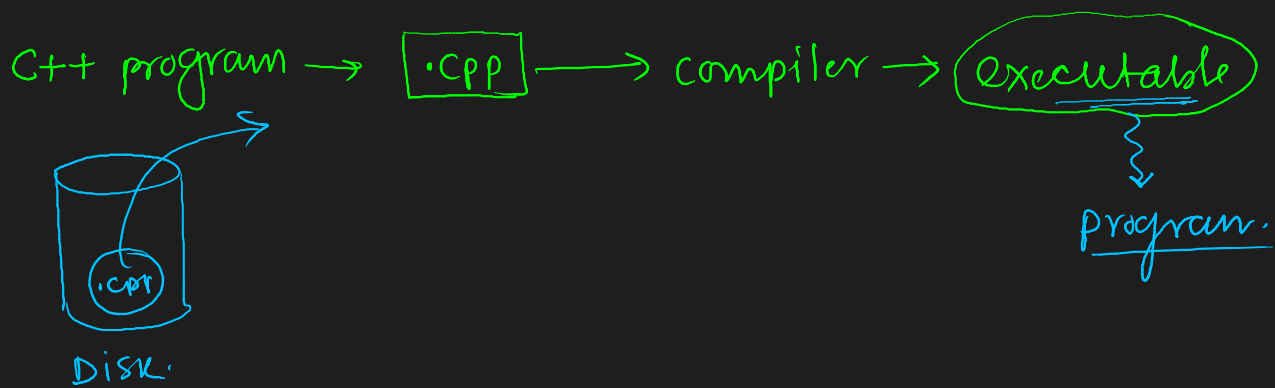
① Cost $\Rightarrow$ Register $\rightarrow$ Cache $\rightarrow$ Main Memory $\rightarrow$ Disk.
Best Material.

② Access Speed $\Rightarrow$ Registers $\rightarrow$ Cache $\rightarrow$ Main Memory $\rightarrow$ Disk.

③ Storage $\Rightarrow$ Disk $\rightarrow$ Main Memory $\rightarrow$ Cache $\rightarrow$ Registers.

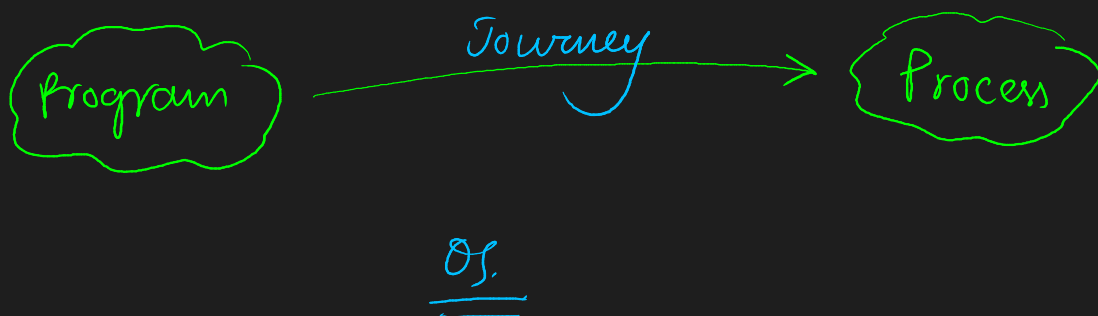
④ Volatility $\Rightarrow$ Primary, Secondary.
Volatile, Non-Volatile.

W-9 Process?



Eg. Mobile $\rightarrow$ {TikTok} $\rightarrow$ OS $\rightarrow$ program $\rightarrow$ process.
 $\rightarrow$ program under execution.

Why Process? $\rightarrow$ way of work $\rightarrow$ user.



How OS creates a process.

① Load the program and static data to memory.

.cpp

used for initialization.

char *name = "Anik"; ~~~~~> Static data.

↳ When the process will be created the name variable will be initialized with "Anik".

② Allocate Runtime Stack.

- Part of memory used for local variables, fn argument & return values.

③ Allocate heap

↳ part of memory used for dynamic allocation.

④ I/O tasks

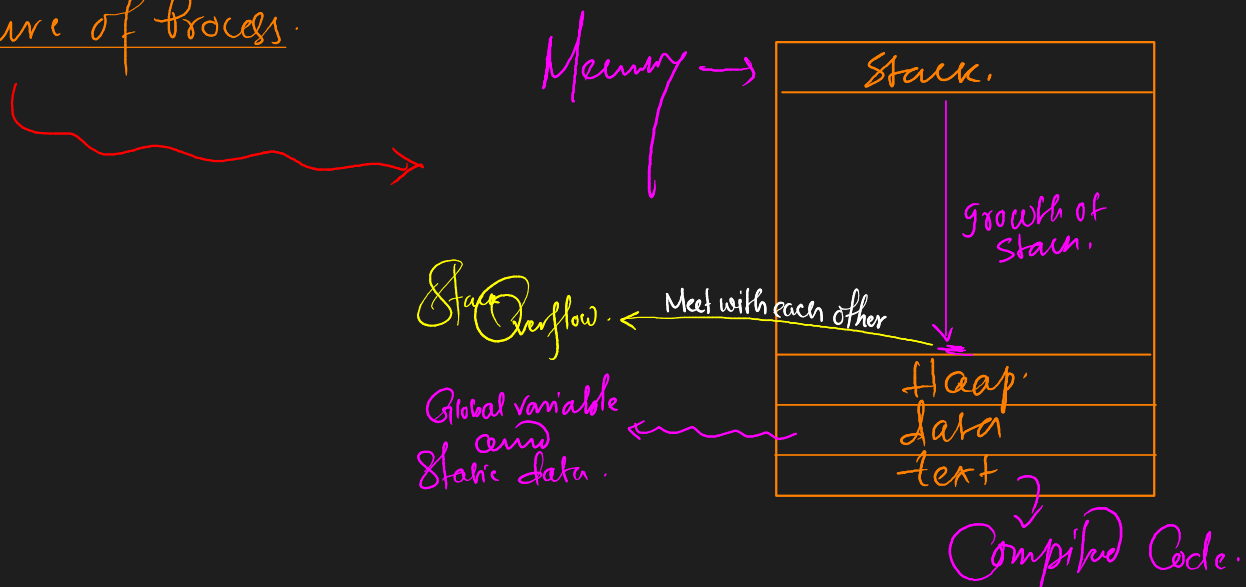
↳ unix → i/p-handle
→ o/p handle
+ error handle.

⑤ OS handoff control to main().

OS found this line → means there is no error before.

os → main() {
return 0;
}

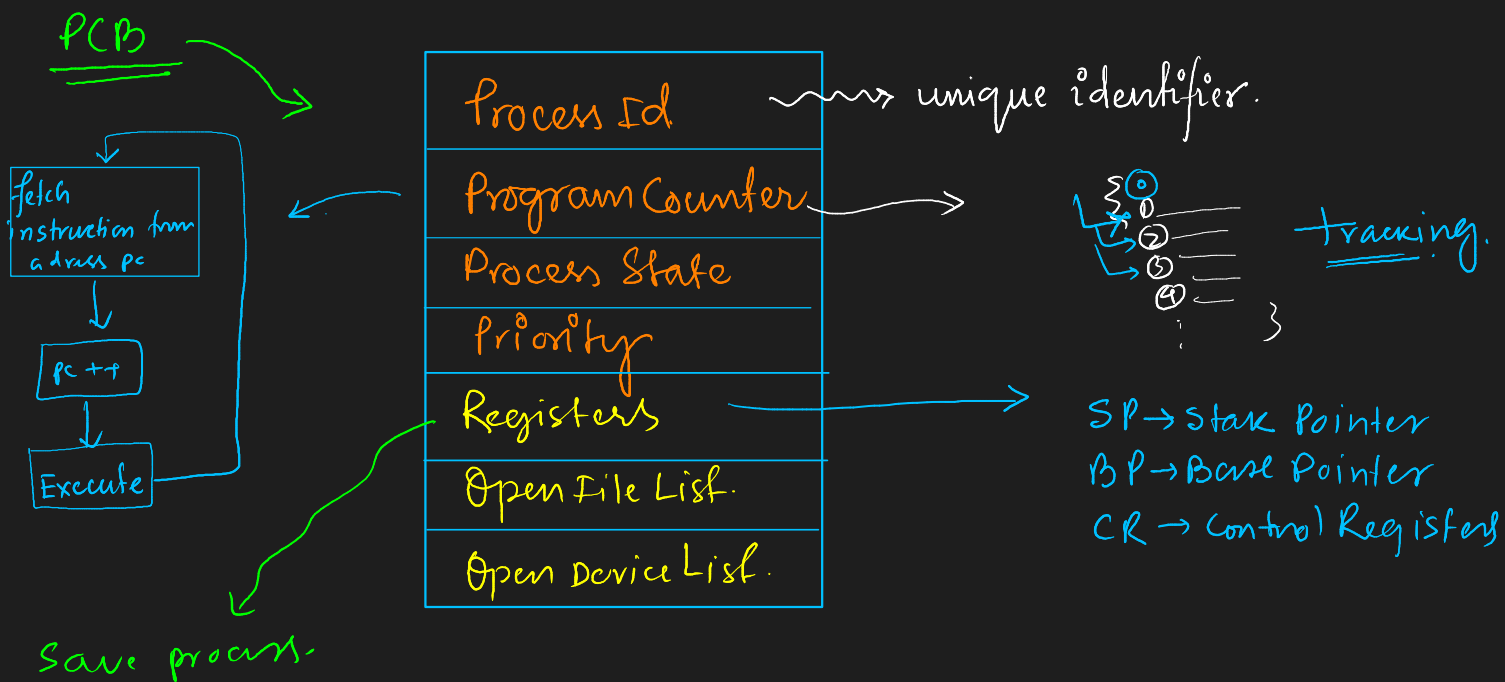
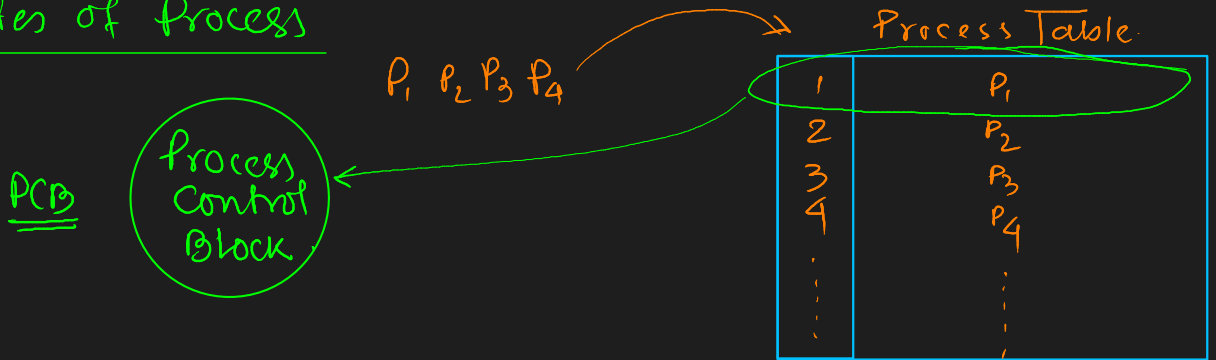
Architecture of Process.



Errors:

- ① Stack Overflow $\rightsquigarrow$ Set base case to handle.
- ② Out of Memory $\rightsquigarrow$ deallocate unnecessary objects.

Attributes of Process

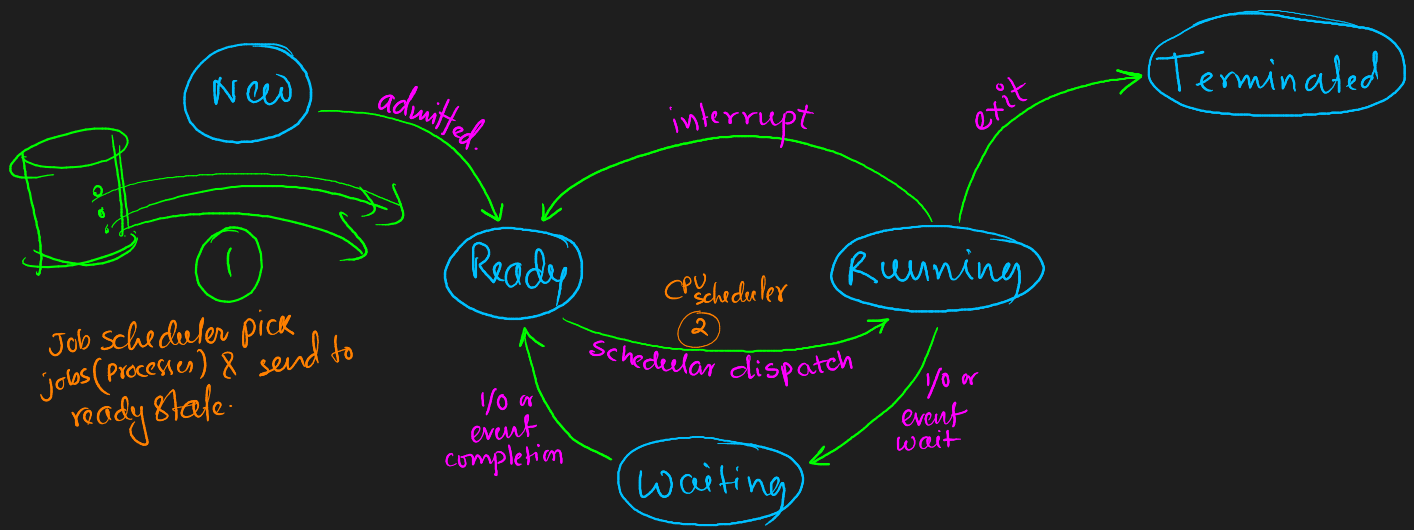


lec-10

Process State

- Lifecycle: (Generation $\rightarrow$ Termination)

- ① **New** $\rightarrow$ program $\rightarrow$ process being done.
- ② **Ready** $\rightarrow$ - Process is in memory.
- Ready Queue.
- ③ **Running** $\rightarrow$ P₁ $\rightarrow$ CPU-Allocate.
- ④ **Waiting** $\rightarrow$ Waiting for I/O complete.
- ⑤ **Terminated** $\rightarrow$ Process finished.



- ① Job Scheduler (LTS) → Long Term Scheduler.
- ② CPU Scheduler (STS) → Short Term Scheduler.

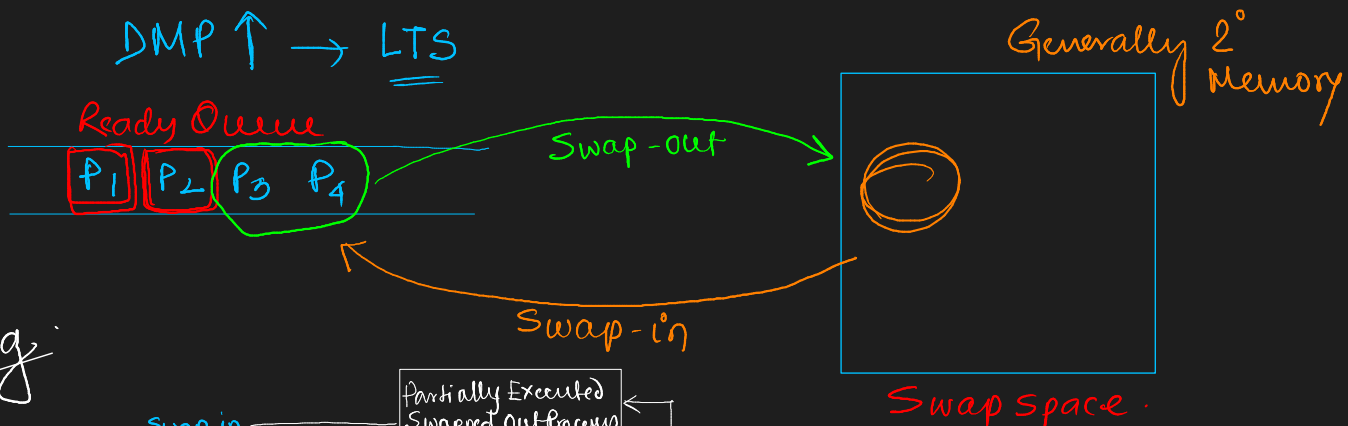
Degree of Multi Programming (DMP) → # processes in the Ready Queue.
(LTS controls this)

Dispatcher → Module of OS that gives control of CPU to a process selected by STS.

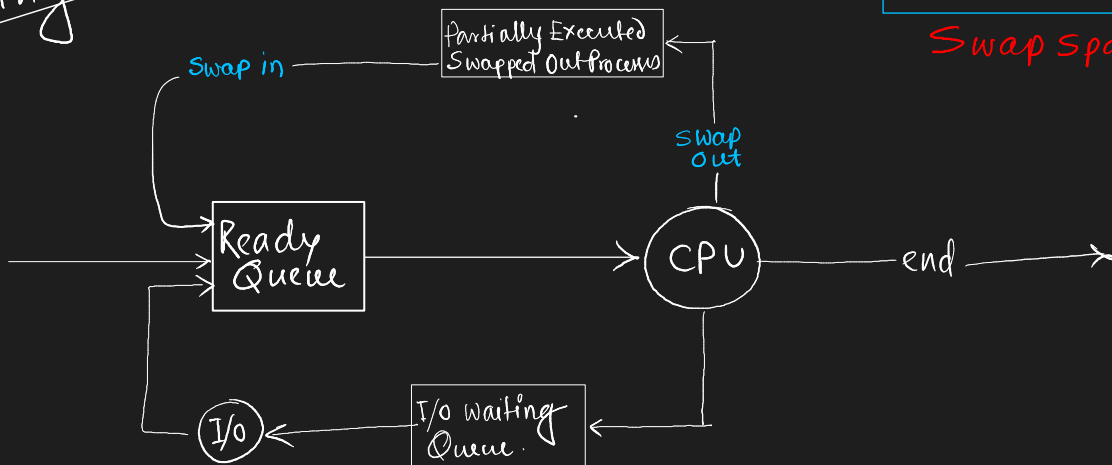
lec-11

MTS → Medium Term Scheduler.

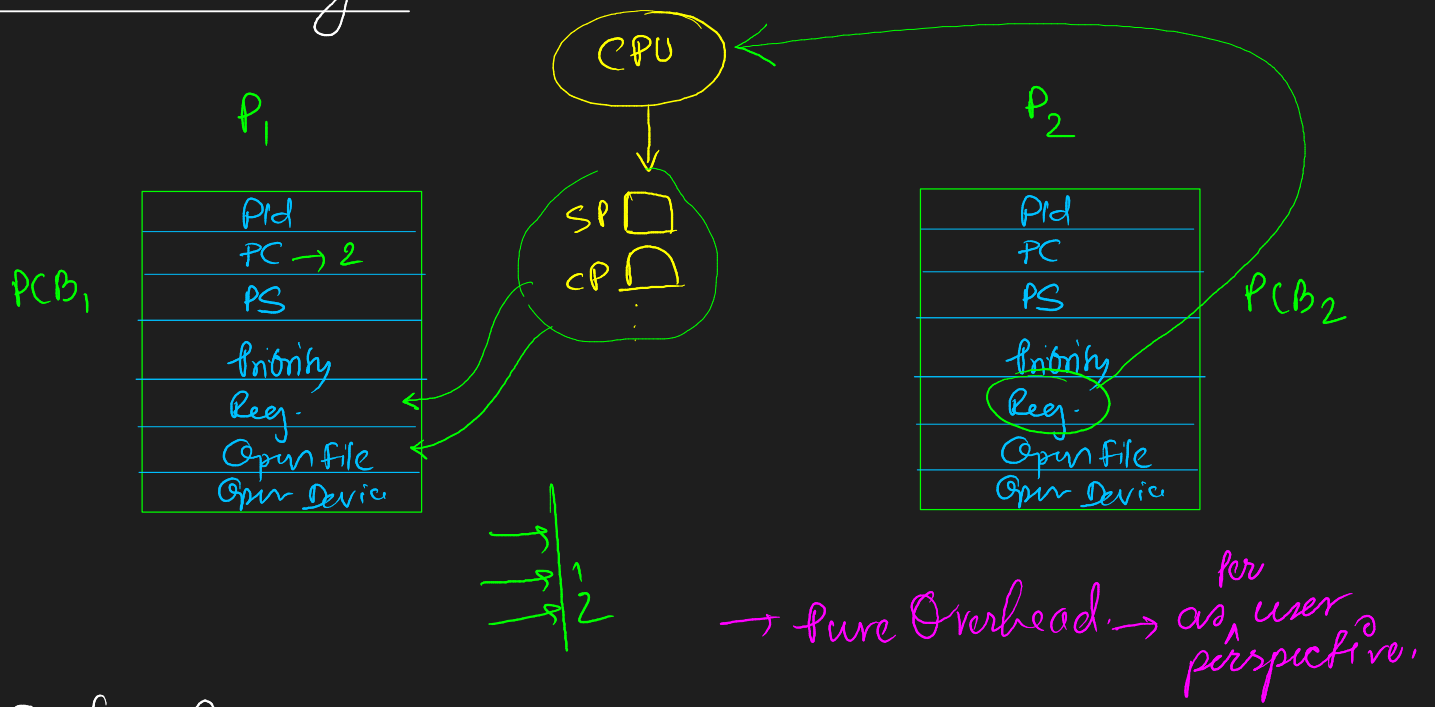
DMP ↑ → LTS



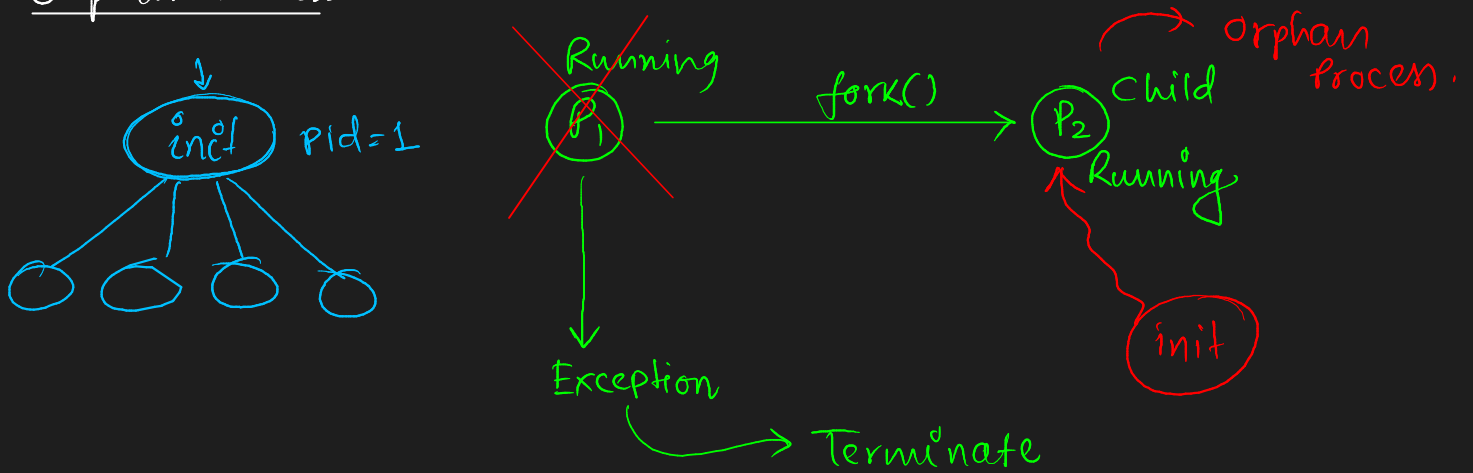
Swapping



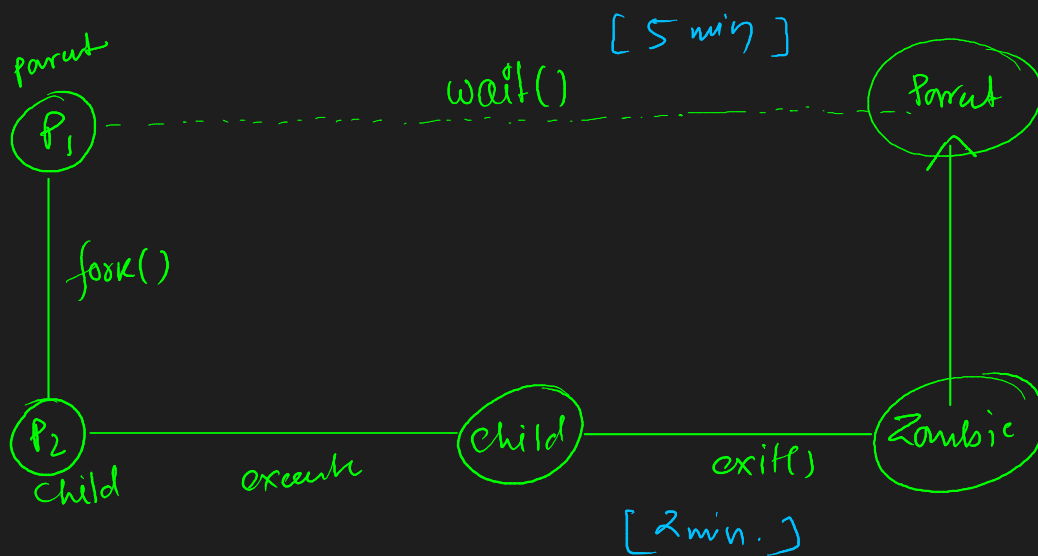
Context Switching



Orphan Process



Zombie Process



practical Example of zombie process ↓

```
▲ pwd
/Users/anikchand/Desktop
▲ bash orphan.sh
▲ vim zombie.sh
▲ bash zombie.sh
zombie.sh: line 2: syntax error near
r unexpected token '('
zombie.sh: line 2: `for i in {1..10
0}`
vim zombie.sh
bash zombie.sh
1m 40s

zombie.sh
#!/bin/bash
1 for i in {1..100}
2 do
3     sleep 16
4 done
5 exec sleep 100
6

ps -la
UID PID PPID F CPU PRI NI SZ RSS WCHAN S ADDR TTY TIME CMD
0 71317 71316 4106 0 31 0 435312656 5104 - Ss 0 ttys000 0:00.02 /usr/bin/login -flp anikchand /bin/
501 71318 71317 4006 0 31 0 435309968 1312 - S 0 ttys000 0:00.45 -zsh
501 71382 1 6 0 31 0 435306384 800 - S 0 ttys000 0:00.00 -zsh
501 71384 1 6 0 31 0 435306368 1072 - S 0 ttys000 0:00.06 -zsh
501 71393 71318 4006 0 31 0 435309008 1072 - S+ 0 ttys000 0:00.02 tmux
501 13802 13801 4006 0 31 0 435311744 4048 - Ss+ 0 ttys001 1:01.51 /bin/zsh
501 13855 1 6 0 31 0 435306384 272 - S 0 ttys001 0:00.00 /bin/zsh
501 13856 1 6 0 31 0 435306368 992 - S 0 ttys001 0:09.03 /bin/zsh
0 20534 20533 106 0 31 0 435309552 288 - Ss+ 0 ttys004 0:00.00 sudo nice -n 10 powermetrics --samp
0 20535 20534 4006 0 36 10 435312928 3632 - SN 0 ttys004 30:23.29 powermetrics --samplers cpu_power g
501 71521 71395 4006 0 31 0 435309968 5328 - Ss+ 0 ttys005 0:01.92 -zsh
501 71602 1 6 0 31 0 435306384 800 - S 0 ttys005 0:00.00 -zsh
501 71603 1 6 0 31 0 435306368 1056 - S 0 ttys005 0:00.06 -zsh
501 71622 71395 4006 0 31 0 435310560 6016 - Ss 0 ttys006 0:01.38 -zsh
501 71672 1 6 0 31 0 435306384 800 - S 0 ttys006 0:00.00 -zsh
501 71673 1 6 0 31 0 435306368 1056 - S 0 ttys006 0:00.06 -zsh
0 74607 71622 4106 0 31 0 435300272 1600 - R+ 0 ttys006 0:00.01 ps -la
0 73115 73114 106 0 31 0 435309552 288 - Ss+ 0 ttys009 0:00.00 sudo nice -n 10 powermetrics --samp
0 73116 73115 4006 0 36 10 435312880 4160 - SN 0 ttys009 0:21.66 powermetrics --samplers cpu_power g

No zombie processes
after execution of
the parent process.
```

